

LINQ GroupBy Method in C# with Examples

In this article, I will discuss the **LINQ GroupBy Method in C#** with Examples. Please read our previous article discussing [LINQ Deferred Execution vs. Immediate Execution in C#](#) with Examples. The LINQ GroupBy Method belongs to the Grouping Operators Category. This method exactly does the same thing as the Group By clause does in SQL Query. As part of this article, we will discuss the following pointers related to the GroupBy Method in C#.

1. [What is LINQ GroupBy Method in C#?](#)
2. [How Do We Use the LINQ GroupBy Method using Query and Method Syntax?](#)
3. [How Do We Use the GroupBy Method and the OrderBy Method in C#?](#)
4. [Multiple Real-Time Examples to Understand LINQ GroupBy Method.](#)
5. [When Should We Use the LINQ GroupBy Method?](#)

What is LINQ GroupBy Method in C#?

The LINQ GroupBy method is a powerful feature in .NET that allows us to organize elements of a collection into groups based on a specified key value. The GroupBy Method groups elements that share a common attribute, allowing us to perform operations on each group. This method is especially useful for aggregating data, performing calculations on subsets of data, and organizing data into a more manageable format.

Each group is represented by an **IGrouping<TKey, TSource>** object, where **TKey** is the type of the key, and **TSource** is the type of the elements in the group. The basic syntax of the GroupBy method looks like this:

```
public static IEnumerable<IGrouping<TKey, TSource>> GroupBy<TSource, TKey>(
    this IEnumerable<TSource> source,
    Func<TSource, TKey> keySelector
)
```

Here,

- **TSource:** The type of the elements of the source sequence.
- **TKey:** The type of the key returned by keySelector.
- **source:** The sequence of elements to group.
- **keySelector:** A function to extract the key for each element.

Examples to Understand LINQ GroupBy Method in C#

Let us understand how to use the LINQ GroupBy Method in C# using both Method and Query Syntax with Examples. We will use the following Student class to understand the GroupBy Method. So, create a class file named **Student.cs** and copy and paste the following code. This class has five properties: **ID, Name, Gender, Branch, and Age**. This class also has one method called **GetStudents()**, which returns a list of all students and will be our data source.

```
using System.Collections.Generic;
namespace GroupByDemo
{
```

```

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public string Branch { get; set; }
    public int Age { get; set; }
    public static List<Student> GetStudents()
    {
        return new List<Student>()
        {
            new Student { ID = 1001, Name = "Preety", Gender = "Female", Branch = "CSE", Age = 20 },
            new Student { ID = 1002, Name = "Snurag", Gender = "Male", Branch = "ETC", Age = 21 },
            new Student { ID = 1003, Name = "Pranaya", Gender = "Male", Branch = "CSE", Age = 21 },
            new Student { ID = 1004, Name = "Anurag", Gender = "Male", Branch = "CSE", Age = 20 },
            new Student { ID = 1005, Name = "Hina", Gender = "Female", Branch = "ETC", Age = 20 },
            new Student { ID = 1006, Name = "Priyanka", Gender = "Female", Branch = "CSE", Age = 21 },
            new Student { ID = 1007, Name = "santosh", Gender = "Male", Branch = "CSE", Age = 22 },
            new Student { ID = 1008, Name = "Tina", Gender = "Female", Branch = "CSE", Age = 20 },
            new Student { ID = 1009, Name = "Celina", Gender = "Female", Branch = "ETC", Age = 22 },
            new Student { ID = 1010, Name = "Sambit", Gender = "Male", Branch = "CSE", Age = 21 }
        };
    }
}

```

Grouping by a Single Property

Now, we want to group the students based on Branch. For a better understanding, please look at the following example, which exactly does the same thing. The following example organizes the students into groups based on their branch (i.e., the branch will act as the key). It also means that students with the same branch will be stored in the same group, where each group has a key and the corresponding student collection. Here, the key will be the Branch, and the collection will be the student belonging to that branch. In the example below, I show how to use the GroupBy Method using both Method Syntax and Query Syntax.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace GroupByDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        //Using Method Syntax
        IEnumerable<IGrouping<string, Student>> GroupByMS =
Student.GetStudents().GroupBy(s => s.Barnch);
        //Using Query Syntax
        IEnumerable<IGrouping<string, Student>> GroupByQS = (from std in
Student.GetStudents()
                                                                group std by
std.Barnch);
        //It will iterate through each groups
        foreach (IGrouping<string, Student> group in GroupByMS)
        {
            Console.WriteLine(group.Key + " : " + group.Count());
            //Iterate through each student of a group
            foreach (var student in group)
            {
                Console.WriteLine("    Name : " + student.Name + ", Age: " +
+ ", Gender : " + student.Gender);
            }
        }
        Console.Read();
    }
}

```

Output:

```

CSE : 6
Name :Preety, Age: 20, Gender :Female
Name :Pranaya, Age: 21, Gender :Male
Name :Anurag, Age: 20, Gender :Male
Name :Priyanka, Age: 21, Gender :Female
Name :santosh, Age: 22, Gender :Male
Name :Tina, Age: 20, Gender :Female
ETC : 4
Name :Snurag, Age: 21, Gender :Male
Name :Hina, Age: 20, Gender :Female
Name :Celina, Age: 22, Gender :Female
Name :Sambit, Age: 21, Gender :Male

```

Note: Each group has a key, and you can access its value by using the key property. Along the same line, you can use the count property to check the number of elements in that group. Again, using a for each loop, you can access all the elements of a group, as shown in the above example.

Grouping Students by Gender in Descending Order, Names in Ascending Order in Each Group

Let us see an example of Understanding the LINQ GroupBy Method with the following requirements.

1. First, Grouping the Students by Gender.
2. Sort the Groups in Descending Order, i.e., sorting the gender in Descending Order.
3. Finally, Sort the Student Names in each group in Ascending Order.

In the following example, we group the students by Gender using the LINQ GroupBy Method. Then, we sort the data by Gender in descending order, and finally, we sort the students in each group by their name in Ascending Order. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            //First Group the Data by Gender
            var GroupByMS = Student.GetStudents().GroupBy(s => s.Gender)
            //Then Sorting the data based on key in Descending
            .OrderByDescending(c => c.Key)
            .Select(std => new
            {
                Key = std.Key,
                //Sorting the Students in Each Group based on
                Ascending order
                Students = std.OrderBy(x => x.Name)
            });
            //Using Query Syntax
            //First Group the Data by Gender
            var GroupByQS = from std in Student.GetStudents()
            //First store the data into a group
            group std by std.Gender into stdGroup
            //Then Sorting the data based on key in Descending
            orderby stdGroup.Key descending
            select new
            {
                Key = stdGroup.Key,
                //Sorting the Students in Each Group based on
                Ascending order
                Students = stdGroup.OrderBy(x => x.Name)
            };
            //It will iterate through each groups
            foreach (var group in GroupByQS)
```

```

    {
        Console.WriteLine(group.Key + " : " + group.Students.Count());
        //Iterate through each student of a group
        foreach (var student in group.Students)
        {
            Console.WriteLine("    Name :" + student.Name + ", Age: " +
+ ", Branch :" + student.Barnch);
        }
    }
    Console.Read();
}
}
}

```

Output:

```

Male : 5
    Name :Anurag, Age: 20, Branch :CSE
    Name :Pranaya, Age: 21, Branch :CSE
    Name :Sambit, Age: 21, Branch :ETC
    Name :santosh, Age: 22, Branch :CSE
    Name :Snurag, Age: 21, Branch :ETC
Female : 5
    Name :Celina, Age: 22, Branch :ETC
    Name :Hina, Age: 20, Branch :ETC
    Name :Preety, Age: 20, Branch :CSE
    Name :Priyanka, Age: 21, Branch :CSE
    Name :Tina, Age: 20, Branch :CSE

```

In the above example, the result is projected to an anonymous type. If you want, you can create a new type with the required properties and project the result to that new type. Let's understand this. First, create a class file named **StudentGroup.cs** and copy and paste the following code.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    public class StudentGroup
    {
        public string Key { get; set; }
        public List<Student> Students { get; set; }
    }
}

```

With the above changes in place, modify the Main method of the Program class as follows: Here, we are projecting the result into the newly created StudentGroup type instead of the anonymous one.

```

using System;

```

```

using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            //First Group the Data by Gender
            var GroupByMS = Student.GetStudents().GroupBy(s => s.Gender)
            //Then Sorting the data based on key in Descending
            .OrderByDescending(c => c.Key)
            .Select(std => new StudentGroup
            {
                Key = std.Key,
                //Sorting the Students in Each Group based on
Ascending order
                Students = std.OrderBy(x => x.Name).ToList()
            });
            //Using Query Syntax
            //First Group the Data by Gender
            var GroupByQS = from std in Student.GetStudents()
            //First store the data into a group
            group std by std.Gender into stdGroup
            //Then Sorting the data based on key in Descending
            orderby stdGroup.Key descending
            select new StudentGroup
            {
                Key = stdGroup.Key,
                //Sorting the Students in Each Group based on
Ascending order
                Students = stdGroup.OrderBy(x => x.Name).ToList()
            };
            //It will iterate through each groups
            foreach (var group in GroupByQS)
            {
                Console.WriteLine(group.Key + " : " + group.Students.Count());
                //Iterate through each student of a group
                foreach (var student in group.Students)
                {
                    Console.WriteLine("    Name :" + student.Name + ", Age: " +
+ ", Branch :" + student.Barnch);
                }
            }
            Console.Read();
        }
    }
}

```

Output:

```

Male : 5
  Name :Anurag, Age: 20, Branch :CSE
  Name :Pranaya, Age: 21, Branch :CSE
  Name :Sambit, Age: 21, Branch :ETC
  Name :santosh, Age: 22, Branch :CSE
  Name :Snurag, Age: 21, Branch :ETC
Female : 5
  Name :Celina, Age: 22, Branch :ETC
  Name :Hina, Age: 20, Branch :ETC
  Name :Preety, Age: 20, Branch :CSE
  Name :Priyanka, Age: 21, Branch :CSE
  Name :Tina, Age: 20, Branch :CSE

```

Using LINQ GroupBy with Aggregation

Let's say you have a collection of Order objects, where each Order has a CustomerId and an Amount. You want to calculate the total amount ordered per customer. First, create the following Order class.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    public class Order
    {
        public int CustomerId { get; set; }
        public decimal Amount { get; set; }
        public static List<Order> GetAllOrders()
        {
            List<Order> orders = new List<Order>
            {
                new Order { CustomerId = 1, Amount = 250 },
                new Order { CustomerId = 2, Amount = 150 },
                new Order { CustomerId = 1, Amount = 100 },
                new Order { CustomerId = 3, Amount = 200 },
                new Order { CustomerId = 2, Amount = 300 }
            };
            return orders;
        }
    }
}

```

Next, modify the Program class as follows. The following example uses GroupBy to group orders by CustomerId and then applies the Sum aggregation function to calculate the total amount ordered per customer.

```

using System;
using System.Collections;

```

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var totalAmountPerCustomer = Order.GetAllOrders()
                .GroupBy(order => order.CustomerId)
                .Select(group => new
                {
                    CustomerId = group.Key,
                    TotalAmount = group.Sum(order => order.Amount)
                });
            foreach (var group in totalAmountPerCustomer)
            {
                Console.WriteLine($"Customer {group.CustomerId} Total Order Ar
{group.TotalAmount}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Customer 1 Total Order Amount: 350
Customer 2 Total Order Amount: 450
Customer 3 Total Order Amount: 200

```

Hierarchical Data Display Example Using Linq GroupBy Method

Hierarchical data organization is a common requirement in software development, where you want to display data in nested categories or levels. LINQ's GroupBy method can be effectively used to create such hierarchical structures by grouping elements of a collection based on specified keys. Let's consider an example where we have a list of Employee objects, and each Employee belongs to a department and has a role within that department. We want to group these employees first by department and then by role within each department, creating a hierarchical display of the data.

Imagine we have a simple Employee class defined as follows, and we have a method returning the collection of employees that we want to group hierarchically.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    class Employee

```

```

{
    public string Name { get; set; }
    public string Department { get; set; }
    public string Role { get; set; }
    public static List<Employee> GetAllEmployees()
    {
        List<Employee> employees = new List<Employee>
        {
            new Employee { Name = "Alice", Department = "IT", Role = "Developer" },
            new Employee { Name = "Bob", Department = "IT", Role = "Tester" },
            new Employee { Name = "Charlie", Department = "HR", Role = "Recruiter" },
            new Employee { Name = "David", Department = "IT", Role = "Developer" },
            new Employee { Name = "Eve", Department = "HR", Role = "Manager" },
            new Employee { Name = "Frank", Department = "IT", Role = "Developer" },
        };
        return employees;
    }
}

```

We can use LINQ's GroupBy method twice to achieve the hierarchical grouping. First, we group by Department, and then within each department group, we group again by Role. For a better understanding, please modify the Program class as follows:

```

using System;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var hierarchicalGrouping = Employee.GetAllEmployees()
                .GroupBy(e => e.Department)
                .Select(departmentGroup => new
                {
                    Department = departmentGroup.Key,
                    Roles = departmentGroup
                        .GroupBy(e => e.Role)
                        .Select(roleGroup => new
                        {
                            Role = roleGroup.Key,
                            Employees = roleGroup.ToList()
                        })
                        .ToList()
                })
                .ToList();
            foreach (var department in hierarchicalGrouping)
            {
                Console.WriteLine($"Department: {department.Department}");
                foreach (var role in department.Roles)

```

```

        {
            Console.WriteLine($"  Role: {role.Role}");
            foreach (var employee in role.Employees)
            {
                Console.WriteLine($"    - {employee.Name}");
            }
        }
    }

    Console.ReadKey();
}
}
}

```

In this example, we first group employees by Department. For each department group, we then create a new grouping by Role. Finally, we iterate over the hierarchical structure to display each department, the roles within that department, and the employees who belong to those roles. When you run the above code, you will get the following output:

```

Department: IT
  Role: Developer
    - Alice
    - David
    - Frank
  Role: Tester
    - Bob
Department: HR
  Role: Recruiter
    - Charlie
  Role: Manager
    - Eve

```

Removing Duplicates Example Using LINQ GroupBy Method

Removing duplicates from a collection is a common task that can be efficiently handled with LINQ in C#. The GroupBy method can be particularly useful for this purpose when you need to consider only certain properties when defining duplicates. Here's how you can use it:

Suppose you have a list of product objects, and each product has properties such as ID, Name, and Category. You want to remove duplicates based on the Name property, ensuring each product name appears only once in the resulting list, regardless of its ID or Category. So, first, create the following Product class.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    class Product
    {

```

```

public int Id { get; set; }
public string Name { get; set; }
public string Category { get; set; }
public static List<Product> GetAllProducts()
{
    List<Product> products = new List<Product>
    {
        new Product { Id = 1, Name = "Table", Category = "Furniture" },
        new Product { Id = 2, Name = "Chair", Category = "Furniture" },
        new Product { Id = 3, Name = "Table", Category = "Office" },
        new Product { Id = 4, Name = "Desk", Category = "Office" },
        new Product { Id = 5, Name = "Chair", Category = "Garden" },
    };
    return products;
}
}
}

```

Next, modify the Program class as follows. In the following example, the GroupBy Method is used to group the products by their Name. Then, for each group, First() is used to select the first product in each group. This effectively removes duplicates based on the Name property because all products in a group have the same name. The resulting list, distinctProductsByName, contains products with unique names.

```

using System;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var distinctProductsByName = Product.GetAllProducts()
                .GroupBy(p => p.Name)
                .Select(g => g.First())
                .ToList();
            foreach (var product in distinctProductsByName)
            {
                Console.WriteLine($"Id: {product.Id}, Name: {product.Name}, Category: {product.Category}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
Id: 1, Name: Table, Category: Furniture
Id: 2, Name: Chair, Category: Furniture
Id: 4, Name: Desk, Category: Office
```

Key Points to Remember:

- The GroupBy method is flexible and allows grouping by multiple criteria if needed. In this example, we grouped by a single property, Name.
- Using First() on each group selects the first item in the group, which is one way to eliminate duplicates. This assumes that any product in the group is an acceptable representative when removing duplicates based on the Name property.
- The resulting list will contain unique items based on the specified property (or properties) used in the GroupBy method. If you need to perform more complex selection logic within each group (e.g., choosing the product with the highest Id), you can replace First() with more sophisticated LINQ methods like OrderByDescending followed by First().

Data Analysis Example Using Linq GroupBy Method

For a data analysis example using the LINQ GroupBy method, let's consider a scenario where we have a collection of sales transactions. Each transaction includes a ProductId, Quantity, and SaleDate. Our goal is to analyze this data to find the total quantity sold for each product by month.

Data Setup

First, let's define a simple Sale class to represent each transaction:

```
using System;
using System.Collections.Generic;
namespace GroupByDemo
{
    public class Sale
    {
        public int ProductId { get; set; }
        public int Quantity { get; set; }
        public DateTime SaleDate { get; set; }
        public static List<Sale> GetAllSales()
        {
            List<Sale> sales = new List<Sale>
            {
                new Sale { ProductId = 1, Quantity = 10, SaleDate = new DateTime(2015, 1, 15) },
                new Sale { ProductId = 1, Quantity = 5, SaleDate = new DateTime(2015, 1, 20) },
                new Sale { ProductId = 2, Quantity = 7, SaleDate = new DateTime(2015, 1, 25) },
                new Sale { ProductId = 1, Quantity = 13, SaleDate = new DateTime(2015, 1, 30) },
            }
        }
    }
}
```

```

        new Sale { ProductId = 2, Quantity = 8, SaleDate = new DateTime(2010, 10, 10) },
        new Sale { ProductId = 1, Quantity = 5, SaleDate = new DateTime(2010, 10, 18) },
        new Sale { ProductId = 2, Quantity = 4, SaleDate = new DateTime(2010, 12, 12) },
        new Sale { ProductId = 3, Quantity = 8, SaleDate = new DateTime(2010, 12, 22) },
        // Add more data as needed
    };
    return sales;
}
}
}

```

Grouping and Analysis

We want to group these sales by ProductId and by the month of the SaleDate, then sum up the Quantity for each group. Here's how you could do it. Please modify the Program class as follows:

```

using System;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var groupedSales = Sale.GetAllSales()
                .GroupBy(sale => new { sale.ProductId, Month = sale.SaleDate.Month })
                .Select(group => new
                {
                    group.Key.ProductId,
                    group.Key.Month,
                    TotalQuantity = group.Sum(sale => sale.Quantity)
                })
                .OrderBy(result => result.ProductId).ThenBy(result => result.Month);
            foreach (var sale in groupedSales)
            {
                Console.WriteLine($"Product {sale.ProductId} - Month: {sale.Month} - Total Quantity Sold: {sale.TotalQuantity}");
            }

            Console.ReadKey();
        }
    }
}

```

Explanation

- **GroupBy:** The data is grouped by both ProductId and the month part of SaleDate. This allows us to analyze sales on a per-product and per-month basis.
- **Select:** We project a new object for each group that includes the ProductId, Month, and the sum of Quantity for that group.
- **OrderBy and ThenBy:** These methods are used to sort the results first by ProductId and then by Month to make the output easier to read.

Expected Output

The output will display the total quantity sold for each product for each month, like the following:

```
Product 1 - Month: 1, Total Quantity Sold: 20
Product 1 - Month: 2, Total Quantity Sold: 13
Product 2 - Month: 1, Total Quantity Sold: 7
Product 2 - Month: 2, Total Quantity Sold: 12
Product 3 - Month: 2, Total Quantity Sold: 8
```

When Should We Use the LINQ GroupBy Method in C#?

Understanding when to use the GroupBy method can help you write more efficient and readable code. Here are several scenarios where GroupBy is particularly useful:

- **Data Aggregation:** When you need to perform aggregate operations (such as counting, summing, averaging, etc.) on a collection of objects based on one or more properties, GroupBy is ideal. For example, you could use it to calculate the total sales for each product category.
- **Organizing Data:** GroupBy is useful for organizing data into a more manageable form. If you have a list of records from a database and you need to display them grouped by a specific field (like date, category, etc.), GroupBy can help you structure your data accordingly.
- **Removing Duplicates:** When you need to remove duplicates based on a specific key or combination of keys, GroupBy can be a handy tool. You can effectively remove duplicates by grouping data by the key(s) and then selecting the first item from each group.
- **Hierarchical Data Display:** For displaying hierarchical data, such as comments and their replies, GroupBy can be used to group comments by their parent ID or some other hierarchical relationship, making it easier to manage and display nested data.
- **Data Analysis:** GroupBy is useful for data analysis tasks, where you might need to categorize data before performing further analysis. It allows you to break down a large dataset into more manageable groups based on specific characteristics.

In the next article, I will discuss the [LINQ GroupBy Method with Multiple Keys in C#](#) with Examples. In this article, I try to explain the **LINQ GroupBy Method in C#** with Examples. I hope you understand the need and how to use the LINQ GroupBy Method in C# and the OrderBy Method with Examples.